Data Visualisation Week 4: Box plots and ridgeline plots

What's happening today?

In the last two weeks, we've looked at how to plot the relationship between two categorical variables (through different kinds of bar charts) and two continuous variables (through different kinds of scatterplots). We've also alluded to relationships between continuous and categorical variables, such as through our graph of how the tempos of Taylor Swift's tracks have changed with their release dates.

Today, we're focusing on how the distributions of continuous variables differ by categorical variables. You saw a bit of this in the first week, when we looked at density curves and boxplots; now we're going to do it a bit more thoroughly.

Getting set up

As with the last few weeks, we're going to start by loading some packages and some data. We're going to install and load a new package, **ggridges**. ggplot2 has loads of different extensions, and ggridges is one such example.

So, let's load some packages…

```
library(tidyverse)
library(ggridges)
library(lubridate)
```

… and let's load some data. (Everything we used last week is still available, I'm just leaving it out so that loading data is quicker!)

```
amy <- read_csv("https://bit.ly/amy-w")
fleet <- read_csv("https://bit.ly/fleetwood-mac-data")
maccas <- read.csv("https://bit.ly/bigmac-data-csv")
```

We're good to go! Let's look at some distributions.

Looking at distributions

We've spent the last few weeks with the Amy Winehouse data. Let's continue this by looking at the distribution of tempo of her tracks.

```
ggplot(data = amy) +
  aes(x = tempo) +
  geom_density()
```

If we want to look at the distribution of a single continuous variable, one approach is to use a density curve. Another is to use a histogram. Let's look at the same data with a histogram.

```
ggplot(data = amy) +
  aes(x = tempo) +
  geom_histogram()
```

Once we run this, we end up with some red text in the console. This is fine, but it might invite us to think about *bins* (effectively, the bars that make up the histogram) – do we want them to be wider or narrower? Do we want more or fewer of them? Let's play around with bins and see what happens. We can play around with them by specifying either the number or the width of the bins, like so.

```
ggplot(data = amy) +
  aes(x = tempo) +
  geom_histogram(bins = 15)
```

```
ggplot(data = amy) +
  aes(x = tempo) +
  geom_histogram(binwidth = 15)
```

Time for a second variable. Are Amy Winehouse's minor tracks slower than her major tracks?

```
ggplot(data = amy) +
  aes(x = tempo, fill = mode) +
  geom_histogram()
```

We've talked about stacked bar charts before, and this seems even more severe. Let's tweak this so that we can compare better.

```
ggplot(data = amy) +
  aes(x = tempo, fill = mode) +
  geom_histogram(bins = 15,
                 position = "identity",
                 alpha = .5)
```

This helps, but has its own issues. Let's try some alternatives.

Back to density curves

Let's try answering that same question – are Amy Winehouse's minor or major tracks quicker? – with some density curves instead.

```
ggplot(data = amy) +
  aes(x = tempo, colour = mode) +
  geom_density()
```

This is a bit more like it. These curves adjust for the fact that there's different numbers of tracks in major and minor keys, and because they're *curves* rather than histograms we can see how they fit together more straightforwardly.

What about albums? Are Amy Winehouse's more recent albums quicker?

```
ggplot(data = amy) +
  aes(x = tempo, colour = album_name) +
  geom_density()
```

This works pretty well for Amy Winehouse's discography, as she only released three albums. However, we could easily imagine how trying to use the same strategy with something like Taylor Swift's albums would be very difficult to read. Comparing two or three overlapping density curves is fairly straightforward to do, but comparing *ten or more* is tricky; it's not as easy to identify the overall trends. As ever, a good way to deal with lots of information in one graph is to turn it into lots of smaller graphs, with the facet_wrap() command.

```
ggplot(data = amy) +
  aes(x = tempo) +
  geom_density() +
  facet_wrap(~ album_name)
```

(Did you ever wonder what the tilde – ~ – was doing in the facet_wrap() parenthesis, when with most other parentheses it's OK to just have the variable name? As an example, let's look at how the tempo of Amy Winehouse's tracks across her albums vary by whether they're in major or minor keys; this means we're faceting by two variables. It also means we're using facet_grid() rather than facet_wrap().)

```
ggplot(data = amy) +
  aes(x = tempo) +
  geom_density() +
  facet_grid(mode ~ album_name, )
```

OK, we're making progress. (This isn't as successful as it could be, though – it's not ideal that we have an NA row taking up loads of space. We'll go through a way to get rid of NA values later on.) Let's also consider two other ways to communicate this information: box plots and violin plots.

```
ggplot(data = amy) +
  aes(x = album_name, y = tempo) +
  geom_boxplot()
```

```
ggplot(data = amy) +
  aes(x = album_name, y = tempo) +
  geom_violin()
```

Astrophysicist and youtuber Angela Collier has a fantastic video about how much she hates violin plots that I very much recommend. You can watch it here (https://www.youtube.com/watch?v=_0QMKFzW9fw): https://www.youtube.com/watch?v=_0QMKFzW9fw (https://www.youtube.com/watch?v=_0QMKFzW9fw)

Moving to Fleetwood Mac

What happens when we're looking at a categorical variable with more than three levels? Fleetwood Mac released a lot of albums, let's look at them.

```
ggplot(data = fleet) +
  aes(x = album, y = tempo) +
  geom_boxplot()
```

This is basically impossible to read because of the number of different albums. One easy way around this problem is just to swap our axes over.

```
ggplot(data = fleet) +
  aes(x = album, y = tempo) +
  geom_boxplot() +
  coord_flip()
```

But we want them in a different order; here, they're just alphabetical. Last week, I showed you the factor() command, which is a powerful way of organising your categorical variables. What I *didn't* show you is that sometimes it's easier. We can reorder our factor levels using the reorder() command. If we want to reorganise our albums based on when they came out, we can do it like so.

```
ggplot(data = fleet) +
  aes(x = fct_reorder(album, ymd(release_date)),
      y = tempo) +
  geom_boxplot() +
  coord_flip()
```

What's happened here? Everything's the same except the first mapping in the aes() parenthesis, so let's look through that.

- we start with fct_reorder(). That indicates we're using a categorical (or *factor*) variable, with levels that are different from the defaults.
- the first thing in the fct_reorder() bracket is the variable we want to put on that axis: album. But while we'd normally close the bracket here, instead we have a comma, anticipating…
- the variable the basis on which we're reordering that categorical variable. Here, we want to reorder according to when the album came out; that information's contained in album_release_date…
- which we need to surround with ymd(), to indicate it's a date of the format YYYY-MM-DD.

Exhausting? But, again, none of these individual bits is that complicated, there's just a lot of them.

And if we want to reorder the categories so that we're reorganising the boxplots from highest to lowest, we can do that with another reorder() command.

```
ggplot(data = fleet) +
  aes(x = fct_reorder(album, tempo),
      y = tempo) +
  geom_boxplot() +
  coord_flip()
```

The difference here is that earlier we were reordering according to release_date; here, we're reordering according to tempo.

Ridgeline plots

(Also known as **joyplots**.)

Did you manage to load ggridges earlier? Let's find out.

```
ggplot(data = fleet) +
  aes(x = tempo,
      y = fct_reorder(album, ymd(release_date))) +
  geom_density_ridges()
```

I really like ridgeline plots, which are a fairly new innovation in data visualisation. I think they make it more

straightforward to compare distributions of a continuous variable across lots of different categories, as in this case. (I like them for fewer categories as well, but I think they're particularly useful here).

However, one issue is that, by default, the y axis goes from smaller numbers at the bottom to bigger numbers at the top. Normally, when we're presenting a chronological list, the earliest items go at the top, and the latest at the bottom; here, we've effectively got the opposite. Let's turn this graph upside down.

```
ggplot(data = fleet) +
  aes(x = tempo,
      y = fct_rev(fct_reorder(album,
                              ymd(release_date)))) +
  geom_density_ridges()
```

This is clumsy, but it works. What we've done is we've wrapped the existing y aesthetic mapping – reorder(album_name, ymd(album_release_date)) – which included a lot more brackets than we're used to, in *yet another* set of brackets, preceded by the **fct_rev()** command. fct_rev() *reverses* the order of levels in a factor, so it's often useful in this kind of context.

We can also reorder from high to low, as we did before.

```
ggplot(data = fleet) +
  aes(x = tempo,
      y = fct_reorder(album, popularity)) +
  geom_density_ridges()
```

Generating new variables

Did you notice some new data at the top? There's some new data this week.

A lot of the time, we can make all the visualisations we want based not only on a single dataset, but also on variables provided in the dataset. Sometimes, though, we need to generate new variables. Let's look at an example.

```
names(maccas)
```

The **maccas** data is what the Economist's Big Mac index is based on; it's a measure of purchasing power in different parts of the world. (You can read more about the Big Mac index, where the data's from, and how it's compiled at this link (https://github.com/TheEconomist/big-mac-data).)

You can see it contains a few different variables: the name of the country, the currency code, the price of a Big Mac in the local currency, the country's GDP in US dollars, the exchange rate between the local currency and the US dollar, and the date of the observation. Maybe we want to know what the relationship is between GDP and the price of Big Macs; are they cheaper in countries with lower GDPs?

One variable that isn't included is the price of a Big Mac in US dollars. A pound sterling's worth (just) more than a US dollar; an Australian dollar's worth less than a US dollar. So comparing prices in local currencies isn't that useful; we need a variable for how much a Big Mac costs in a single currency. And we might as well use US dollars (though we don't have to).

Let's start by generating a new variable, using the **mutate()** command.

```
maccas %>%
  mutate(price_dollars = local_price/dollar_ex) %>%
  ggplot() +
  aes(x = GDP_dollar, y = price_dollars) +
  geom_point()
```

This is similar to what we've seen before; we've started with data, added a pipe, and specified what we want to do to the data. Here, we want a new variable, which I've called *price_dollars*: I've generated this by dividing the local price by the dollar exchange rate. The forward slash - / - means *divide by*; you can add variables with +, subtract with -, and multiply with *.

A problem here, though, is the number of observations. What if we just want the *current* relationship? To do this, we can limit our observations to the most recent wave of data, by going back to filter():

```
maccas %>%
  mutate(price_dollars = local_price/dollar_ex) %>%
  filter(date == "2022-07-01") %>%
  ggplot() +
  aes(x = GDP_dollar, y = price_dollars) +
  geom_point()
```

I might not get time for this bit

What if we wanted to know how this relationship had changed over time? An obvious starting point is to facet by date.

```
maccas %>%
  mutate(price_dollars = local_price/dollar_ex) %>%
  ggplot() +
  aes(x = GDP_dollar, y = price_dollars) +
  geom_point() +
  facet_wrap(~ date)
```

There's an obvious problem here, though. Not all the observations include all the data we want, which is why loads of our facets are empty. To get around this problem, we can use **drop_na**: this means we're just looking at *complete cases* for our data, rather than cases with missing data. Here's how we use it.

```
maccas %>%
  drop_na(GDP_dollar, local_price, dollar_ex) %>%
  mutate(price_dollars = local_price/dollar_ex) %>%
  ggplot() +
  aes(x = GDP_dollar, y = price_dollars) +
  geom_point() +
  facet_wrap(~ date)
```

What is important here is that we included the variables we are using in our visualisation in our drop_na function as arguments. We *could* just use drop_na() without any arguments here, which would drop all data with *any* missing values — however, we generally don't want to get into the practice of throwing away perfectly good data (data that has values for all of the variables we *are* interested in).

Task

One task this week, but it's a tricky one.

Please draw a *ridgeline plot* of the prices of a Big Mac in each country in the world, expressed as price in dollars as a fraction of GDP per head in dollars in each of those countries, sorted highest-to-lowest. So I want to see a graph of the range of those values in each country over the period that the Economist had data for, with the country where a Big Mac costs the largest proportion of GDP per head at the top, and the country where a Big Mac costs the largest proportion of GDP per head at the bottom.

Please also show me something interesting about how a continuous variable varies by a categorical variable in one of the bits of Spotify data, and post it to the discussion board before next week's session.